

GOVERNMENT POLYTECHNIC
JAMNAGAR



MICRO PROJECT
ON

**“CRIME REGISTER AND RECORD
MANAGEMENT SYSTEM”**

**SUBJECT: ADVANCED OBJECT-ORIENTED
PROGRAMMING**

CODE: 4340701

SUBMITTED BY

VAIBHAV MAKWANA

236250307062

VEER PARMAR

236250307085

UNDER GUIDENCE OF: Ms. Jaynika Vadalia

Academic Year: EVEN 2025-26

1. Introduction

The Criminal Record Management System is a console-based application developed in Java for storing, managing, and retrieving criminal records. It allows users to add, update, delete, and search criminal records efficiently. The application demonstrates object-oriented programming concepts such as classes, objects, inheritance, encapsulation, and abstraction.

2. Objectives

The main objectives of the **Criminal Record Management System** are:

- To digitally store and manage criminal records in a structured format.
- To reduce manual errors and delays in accessing criminal information.
- To provide easy search, update, and delete functionalities for records.
- To implement core Object-Oriented Programming (OOP) concepts in Java.
- To ensure data is handled securely and efficiently within the system.

3. Features

The *Criminal Record Management System* offers several core features that make criminal data handling efficient and secure. Below are the key features:

- **Add New Criminal Record:** Allows users to input new criminal details such as name, age, gender, crime committed, date of arrest, and more.
- **View All Records:** Displays a list of all stored records with neatly formatted data for easy analysis and review.
- **Search Functionality:** Enables users to search for a particular criminal using criteria like name or crime type.
- **Update Existing Records:** Records can be modified if any detail changes or needs correction.
- **Delete Records:** Enables the removal of outdated or incorrect data from the system.
- **User-Friendly Console Interface:** Built using simple and intuitive Java console-based UI to interact with the system smoothly.
- **Data Storage in Files:** Criminal data is stored and managed using file handling, ensuring information persists between sessions.

- **Efficient Navigation:** Menu-driven design helps users navigate through the system effortlessly.

.

4. Application Structure

The **Criminal Record Management System** is a console-based Java application developed using the principles of Object-Oriented Programming (OOP). The structure of the application is modular and organized into distinct components to promote clarity, maintainability, and scalability.

Main Components:

- **1. Main.java**
 - Acts as the entry point of the application.
 - Displays the main menu and handles user input.
 - Connects all functionalities like add, view, search, update, and delete.
- **2. Criminal.java**
 - A class that defines the structure of a criminal record.
 - Contains private fields like name, age, gender, crime, arrestDate, etc.
 - Includes constructors, getters, setters, and toString() method.
- **3. CriminalManager.java**
 - Handles the core logic of managing records.
 - Implements file handling to store, retrieve, and update records in text files.

- Contains methods such as addCriminal(), viewCriminals(), searchCriminal(), updateCriminal(), and deleteCriminal().
- **4. FileHandler.java**
 - A utility class that simplifies reading and writing files.
 - Ensures separation of concerns by abstracting file operations from business logic.

File Storage:

- All criminal data is saved in a structured format within a plain text file (criminals.txt), allowing persistent storage without the use of databases.

Flow of Execution:

1. Application starts from Main.java.
2. User selects an option from the menu.
3. The chosen action is handled by CriminalManager.
4. Criminal.java is used to model data.
5. File operations are handled through Java's File I/O classes.

6. Conclusion

The Criminal Record Management System project helped me understand how to use Java and Object-Oriented Programming to build real-life applications. I learned how to create and manage records using classes, objects, and file handling. This project improved my coding skills and gave me confidence to make more advanced systems in the future.

Source Code:-

```
import java.util.Scanner;
import java.util.Random;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.io.*;

public class MP {
    Scanner sc = new Scanner(System.in);
    String criminalName;
    int age;
    String crime;
    String gender;
    String address;
    final String FILE_NAME = "criminal_records.txt";

    // Method to register a new case and store details in a file
    void new_case() {
        System.out.println("Enter Criminal name: ");
        criminalName = sc.nextLine();

        System.out.println("Enter the age: ");
        age = sc.nextInt();
        sc.nextLine(); // Consume leftover newline

        System.out.println("Enter gender of Criminal: ");
        gender = sc.nextLine();

        System.out.println("Enter the address: ");
        address = sc.nextLine();

        System.out.println("Enter the Crime: ");
        crime = sc.nextLine();

        saveToFile();
        System.out.println("\nCase Registered Successfully and Saved to File!");
    }

    // Method to save case details to a file
    void saveToFile() {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(FILE_NAME, true))) {
            writer.write(criminalName + "," + age + "," + gender + "," + address + "," + crime);
            writer.newLine();
        } catch (IOException e) {
            System.out.println("Error saving data: " + e.getMessage());
        }
    }

    //-----
    // Method to update details in the file
    void updateDetails() {
        if (!new File("").exists()) {
            System.out.println("\nNo record found! Please register a case first.");
            return;
        }

        System.out.println("Enter the name of the criminal to update: ");
        String searchName = sc.nextLine();
        boolean found = false;
```

```

try {
    File inputFile = new File("criminal_records.txt");
    File tempFile = new File("temp.txt");

    BufferedReader reader = new BufferedReader(new FileReader(inputFile));
    BufferedWriter writer = new BufferedWriter(new FileWriter(tempFile));

    String currentLine;
    while ((currentLine = reader.readLine()) != null) {
        String[] details = currentLine.split(",");
        if (details[0].equalsIgnoreCase(searchName)) {
            found = true;
            System.out.println("\nUpdating details for: " + searchName);
            System.out.println("1. Update Name");
            System.out.println("2. Update Age");
            System.out.println("3. Update Gender");
            System.out.println("4. Update Address");
            System.out.println("5. Update Crime");
            System.out.println("6. Cancel");
            System.out.print("Enter your choice: ");

            int updateChoice = sc.nextInt();
            sc.nextLine(); // Consume newline

            switch (updateChoice) {
                case 1:
                    System.out.print("Enter new name: ");
                    details[0] = sc.nextLine();
                    break;
                case 2:
                    System.out.print("Enter new age: ");
                    details[1] = String.valueOf(sc.nextInt());
                    sc.nextLine();
                    break;
                case 3:
                    System.out.print("Enter new gender: ");
                    details[2] = sc.nextLine();
                    break;
                case 4:
                    System.out.print("Enter new address: ");
                    details[3] = sc.nextLine();
                    break;
                case 5:
                    System.out.print("Enter new crime details: ");
                    details[4] = sc.nextLine();
                    break;
                case 6:
                    System.out.println("Update canceled.");
                    return;
                default:
                    System.out.println("Invalid choice! Please try again.");
                    return;
            }

            writer.write(String.join(",", details));
            writer.newLine();
            System.out.println("\nCriminal details updated successfully!");
        } else {
            writer.write(currentLine);
            writer.newLine();
        }
    }
}

```

```

    }
}

reader.close();
writer.close();

// Replace old file with updated one
inputFile.delete();
tempFile.renameTo(inputFile);

if (!found) {
    System.out.println("\nCriminal not found in records.");
}

} catch (IOException e) {
    System.out.println("Error updating data: " + e.getMessage());
}
}

//-----
// Method to generate a random court hearing date
// void courtHiring() {
//     if (!new File("criminal_records.txt").exists()) {
//         System.out.println("\nNo case registered yet! Please register a case first.");
//         return;
//     }

//     Random rand = new Random();
//     LocalDate today = LocalDate.now();
//     int daysToAdd = rand.nextInt(30) + 1; // Random court date within the next 30 days
//     LocalDate courtDate = today.plusDays(daysToAdd);

//     DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd MMMM yyyy");
//     String formattedDate = courtDate.format(formatter);

//     System.out.println("\nThe court hearing has been scheduled on: " + formattedDate);
// }
// Method to generate a court hearing date for a specific criminal
void courtHiring() {
    if (!new File(FILE_NAME).exists()) {
        System.out.println("\nNo case registered yet! Please register a case first.");
        return;
    }

    System.out.println("Enter the name of the criminal: ");
    String searchName = sc.nextLine();
    boolean found = false;

    try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
        String currentLine;
        while ((currentLine = reader.readLine()) != null) {
            String[] details = currentLine.split(",");
            if (details[0].equalsIgnoreCase(searchName)) {
                found = true;

                // Generate a random court date
                Random rand = new Random();
                LocalDate today = LocalDate.now();
                int daysToAdd = rand.nextInt(30) + 1; // Random date within the next 30 days
                LocalDate courtDate = today.plusDays(daysToAdd);

                DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd MMMM yyyy");

```

```

        String formattedDate = courtDate.format(formatter);

        System.out.println("\nThe court hearing for " + searchName + " is scheduled on: " +
formattedDate);
        break;
    }
}

if (!found) {
    System.out.println("\nCriminal not found in records.");
}

} catch (IOException e) {
    System.out.println("Error reading data: " + e.getMessage());
}
}

//-----
void adminPanel() {
    final String ADMIN_KEY = "nothing"; // Set your admin access key

    System.out.print("Enter Admin Key: ");
    String inputKey = sc.nextLine();

    if (!inputKey.equals(ADMIN_KEY)) {
        System.out.println("Access Denied! Incorrect Key.");
        return;
    }

    int choice;
    while (true) {
        System.out.println("\nAdmin Panel");
        System.out.println("1. Display All Criminals");
        System.out.println("2. Close Case (Remove from Records)");
        System.out.println("3. Filter Cases by Crime Category");
        System.out.println("4. Search Criminal by Name");
        System.out.println("5. Basic Report Generation ");
        System.out.println("6. Exit Admin Panel");
        System.out.print("Enter your choice: ");

        choice = sc.nextInt();
        sc.nextLine(); // Consume newline

        switch (choice) {
            case 1:
                displayAllCriminals();
                break;
            case 2:
                closeCase();
                break;
            case 3:
                filterByCrimeCategory();
                break;
            case 4:
                searchCriminalByName();
                break;
            case 5:
                generateReport();
                break;
            case 6:
                System.out.println("Exiting Admin Panel...");

```

```

        return;
    default:
        System.out.println("Invalid choice! Please try again.");
    }
}
}

//-----
// Function to display all criminals
void displayAllCriminals() {
    File file = new File("criminal_records.txt");
    if (!file.exists()) {
        System.out.println("No records found!");
        return;
    }

    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        System.out.println("\nList of Criminals:");
        while ((line = reader.readLine()) != null) {
            String[] details = line.split(",");
            System.out.println("Name: " + details[0] + ", Age: " + details[1] +
                ", Gender: " + details[2] + ", Address: " + details[3] + ", Crime: " + details[4]);
        }
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }
}

//-----
// Function to remove a case from the file (Close Case)
// Method to close a case by marking it as "closed" in the file
void closeCase() {
    if (!new File(FILE_NAME).exists()) {
        System.out.println("\nNo case records found!");
        return;
    }

    System.out.print("Enter the name of the criminal whose case you want to close: ");
    String searchName = sc.nextLine();
    boolean found = false;

    try {
        File inputFile = new File(FILE_NAME);
        File tempFile = new File("temp.txt");

        BufferedReader reader = new BufferedReader(new FileReader(inputFile));
        BufferedWriter writer = new BufferedWriter(new FileWriter(tempFile));

        String currentLine;
        while ((currentLine = reader.readLine()) != null) {
            String[] details = currentLine.split(",");

            if (details[0].equalsIgnoreCase(searchName)) {
                found = true;
                if (details.length == 5) { // Append "closed" status if not present
                    writer.write(currentLine + ",closed");
                } else {
                    details[5] = "closed"; // Update existing status to "closed"
                    writer.write(String.join(",", details));
                }
                System.out.println("\nCase for " + searchName + " has been closed.");
            } else {

```

```

        writer.write(currentLine);
    }
    writer.newLine();
}

reader.close();
writer.close();

// Replace the old file with updated data
inputFile.delete();
tempFile.renameTo(inputFile);

if (!found) {
    System.out.println("\nCriminal not found in records.");
}

} catch (IOException e) {
    System.out.println("Error updating data: " + e.getMessage());
}
}

//-----
// Function to filter cases by crime category
void filterByCrimeCategory() {
    System.out.print("Enter crime category to filter: ");
    String crimeCategory = sc.nextLine();
    boolean found = false;

    File file = new File("criminal_records.txt");
    if (!file.exists()) {
        System.out.println("No records found!");
        return;
    }

    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        System.out.println("\nCriminals involved in " + crimeCategory + ":");
        while ((line = reader.readLine()) != null) {
            String[] details = line.split(",");
            if (details[4].equalsIgnoreCase(crimeCategory)) {
                System.out.println("Name: " + details[0] + ", Age: " + details[1] +
                    ", Gender: " + details[2] + ", Address: " + details[3]);
                found = true;
            }
        }
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }

    if (!found) {
        System.out.println("No records found for this crime category.");
    }
}

//-----
// Function to search for a criminal by name
void searchCriminalByName() {
    System.out.print("Enter the name of the criminal to search: ");
    String searchName = sc.nextLine();
    boolean found = false;

    File file = new File(FILE_NAME);

```

```

    if (!file.exists()) {
        System.out.println("No records found!");
        return;
    }

    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {
            String[] details = line.split(",");
            if (details[0].equalsIgnoreCase(searchName)) {
                System.out.println("\nCriminal Found:");
                System.out.println("Name: " + details[0] + ", Age: " + details[1] +
                    ", Gender: " + details[2] + ", Address: " + details[3] + ", Crime: " + details[4]);
                found = true;
                break;
            }
        }
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }

    if (!found) {
        System.out.println("Criminal not found in records.");
    }
}

//-----
void generateReport() {
    if (!new File(FILE_NAME).exists()) {
        System.out.println("\nNo case records found!");
        return;
    }

    int totalCrimes = 0;
    int openCases = 0;
    int closedCases = 0;

    try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
        String currentLine;
        while ((currentLine = reader.readLine()) != null) {
            totalCrimes++;
            String[] details = currentLine.split(",");
            if (details.length > 5 && details[5].equalsIgnoreCase("closed")) {
                closedCases++;
            } else {
                openCases++;
            }
        }
    } catch (IOException e) {
        System.out.println("Error reading data: " + e.getMessage());
    }

    System.out.println("\n===== Crime Report =====");
    System.out.println("Total Crimes Registered: " + totalCrimes);
    System.out.println("Open Cases: " + openCases);
    System.out.println("Closed Cases: " + closedCases);
}

//-----
// Method to generate a detailed report for a specific criminal
void generateCriminalReport() {
    if (!new File(FILE_NAME).exists()) {
        System.out.println("\nNo case records found!");
    }
}

```

```

        return;
    }

    System.out.print("Enter the name of the criminal for the report: ");
    String searchName = sc.nextLine();
    boolean found = false;

    try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
        String currentLine;
        while ((currentLine = reader.readLine()) != null) {
            String[] details = currentLine.split(",");

            if (details[0].equalsIgnoreCase(searchName)) {
                found = true;
                System.out.println("\n=====");
                System.out.println("    CRIMINAL REPORT");
                System.out.println("=====");
                System.out.println("-> Name      : " + details[0]);
                System.out.println("-> Age       : " + details[1]);
                System.out.println("-> Gender    : " + details[2]);
                System.out.println("-> Address   : " + details[3]);
                System.out.println("-> Crime     : " + details[4]);
                if (details.length > 5) {
                    System.out.println("Case Status: " + details[5]); // Displays if case is closed
                } else {
                    System.out.println("Case Status: Open");
                }
                System.out.println("=====\\n");
                break;
            }
        }
    } catch (IOException e) {
        System.out.println("Error reading data: " + e.getMessage());
    }

    if (!found) {
        System.out.println("\\nCriminal not found in records.");
    }
}

//-----
public static void main(String[] args) {
    MP obj = new MP();
    Scanner sc = new Scanner(System.in);
    int choice;

    while (true) {
        System.out.println("\\nCriminal Investigation System");
        System.out.println("1. Register a New Case");
        System.out.println("2. Update Details of Criminal");
        System.out.println("3. Court Hearing");
        System.out.println("4. Admin Panel");
        System.out.println("5. Criminal Report");
        System.out.println("6. Exit");
        System.out.print("Enter your choice: ");

        choice = sc.nextInt();
        sc.nextLine(); // Consume leftover newline

        switch (choice) {
            case 1:

```

```

        obj.new_case();
        break;
    case 2:
        obj.updateDetails();
        break;
    case 3:
        obj.courtHiring();
        break;
    case 4:
        obj.adminPanel();
        break;
    case 5:
        obj.generateCriminalReport();
        break;
    case 6:
        System.out.println("Exiting... Goodbye!");
        sc.close();
        return;
    default:
        System.out.println("Wrong choice! Please try again.");
        break;
    }
}
}
}

```

OUTPUT:-

```

Crime Investigation System
1. Register a New Case
2. Update Details of Criminal
3. Court Hearing
4. Admin Panel
5. Criminal Report
6. Exit
Enter your choice:

```

• Register New Case

```

Enter your choice: 1
Enter Criminal name:
ABCD
Enter the age:
25
Enter gender of Criminal:
Male
Enter the address:
Jamnagar
Enter the Crime:
Theft

```

• Court Hearing Date

```
Crime Investigation System
```

1. Register a New Case
2. Update Details of Criminal
3. Court Hearing
4. Admin Panel
5. Criminal Report
6. Exit

```
Enter your choice: 3
```

```
Enter the name of the criminal:
```

```
ABCD
```

```
The court hearing for ABCD is scheduled on: 11 April 2025
```

• Access Admin Panel

```
Enter your choice: 4
```

```
Enter Admin Key: nothing
```

• Admin Panel with Choice 1

```
Admin Panel
```

1. Display All Criminals
2. Close Case (Remove from Records)
3. Filter Cases by Crime Category
4. Search Criminal by Name
5. Basic Report Generation
6. Exit Admin Panel

```
Enter your choice: 1
```

```
List of Criminals:
```

```
Name: ABC, Age: 18, Gender: m, Address: Jamnagar, Crime: Theft
```

```
Name: BCD, Age: 25, Gender: F, Address: Rape, Crime: Rape
```

```
Name: ABCD, Age: 25, Gender: Male, Address: Jamnagar, Crime: Theft
```

```
Name: BCDE, Age: 26, Gender: F, Address: Jamnagare, Crime: Robbrey
```

• Close Case

```
Enter your choice: 2
```

```
Enter the name of the criminal whose case you want to close: ABC
```

```
Case for ABC has been closed.
```

- Filter Crime By Category

```
Enter your choice: 3
Enter crime category to filter: theft

Criminals involved in theft:
Name: ABC, Age: 18, Gender: m, Address: Jamnagar
Name: ABCD, Age: 25, Gender: Male, Address: Jamnagar
Abhishek, Age: 20, Gender: Male, Address: Jamnagar
```

- Search By Criminal Name

```
Enter your choice: 4
Enter the name of the criminal to search: ABCD

Criminal Found:
Name: ABCD, Age: 25, Gender: Male, Address: Jamnagar, Crime: Theft
```

- Basic Crime Report (Total Crime)

```
Enter your choice: 5

===== Crime Report =====
Total Crimes Registered: 4
Open Cases: 3
Closed Cases: 1
```

- Criminal Report Generation

```
Enter your choice: 5
Enter the name of the criminal for the report: ABCD

=====
CRIMINAL REPORT
=====
-> Name      : ABCD
-> Age       : 25
-> Gender    : Male
-> Address   : Jamnagar
-> Crime     : Theft
Case Status: Open
=====
```